

# Circuit to Code

Hands-on electronics lessons toward programming and embedded projects.

Version 0.4.0

Copyright 2026 Lalatendu Mohanty. Licensed under Apache License 2.0.

Source and updates: <https://github.com/LalatenduMohanty/circuit-to-code>

## Contents

---

|                                                |           |
|------------------------------------------------|-----------|
| <b>Learning Scratch Programming.....</b>       | <b>4</b>  |
| Why learn Scratch? .....                       | 4         |
| Getting started .....                          | 5         |
| Core skills (watch these first).....           | 5         |
| Beginner projects.....                         | 6         |
| Intermediate projects .....                    | 6         |
| Advanced projects .....                        | 7         |
| What's next.....                               | 7         |
| <b>LEGO SPIKE Prime (optional) .....</b>       | <b>8</b>  |
| Why try SPIKE after Scratch? .....             | 8         |
| What you need .....                            | 8         |
| How to learn (use official lessons).....       | 8         |
| Learning outline (checklist).....              | 9         |
| Scratch → SPIKE reminder .....                 | 9         |
| What's next.....                               | 9         |
| <b>Beginner Circuits.....</b>                  | <b>10</b> |
| What you need .....                            | 10        |
| Mission 0: The Maker Rules .....               | 12        |
| If it doesn't work (Troubleshoot) .....        | 14        |
| Phase 1: The Basics of Flow .....              | 14        |
| Phase 2: Sensors and Adjustments.....          | 18        |
| Phase 3: Hardware Magic (Logic & Timers) ..... | 20        |
| What's next.....                               | 24        |
| <b>Intermediate Circuits.....</b>              | <b>25</b> |
| What you need .....                            | 25        |
| Mission 0: Intermediate Maker Rules .....      | 26        |
| Phase 1: Stronger Switching .....              | 26        |

|                                                      |           |
|------------------------------------------------------|-----------|
| Phase 2: Signals Boards Can Read .....               | 28        |
| Phase 3: Motors and Speed .....                      | 30        |
| Mission wrap: Power checklist (before Arduino) ..... | 32        |
| What's next .....                                    | 32        |
| <b>Guide to micro:bit V2 .....</b>                   | <b>33</b> |
| 1. Getting your gear .....                           | 33        |
| 2. Micro:bit vs. Arduino .....                       | 35        |
| 3. Extra electronic components .....                 | 35        |
| 4. Setting up your lab .....                         | 36        |
| 5. Video tutorials and project ideas .....           | 36        |
| 6. Other starter kits .....                          | 37        |
| What's next .....                                    | 37        |
| <b>Guide to CircuitPython .....</b>                  | <b>38</b> |
| 1. Getting your gear .....                           | 38        |
| 2. Why CircuitPython after Scratch / MakeCode? ..... | 39        |
| 3. Install CircuitPython (Pico) .....                | 39        |
| 4. Scratch / MakeCode → Python map .....             | 40        |
| 5. Wiring habits (same as Intermediate) .....        | 40        |
| 6. Curated projects .....                            | 41        |
| 7. Learning checklist .....                          | 41        |
| What's next .....                                    | 41        |
| <b>Guide to Arduino .....</b>                        | <b>42</b> |
| 1. Getting your gear .....                           | 42        |
| 2. Arduino vs. micro:bit .....                       | 43        |
| 3. Install and first upload .....                    | 43        |
| 4. Scratch / MakeCode → sketch map .....             | 43        |
| 5. Wiring habits (do not skip) .....                 | 44        |
| 6. Curated projects (build these next) .....         | 45        |
| 7. Learning checklist .....                          | 45        |
| What's next .....                                    | 46        |
| <b>Robot Missions .....</b>                          | <b>47</b> |
| You're building mission robots .....                 | 47        |
| Two ways to practice .....                           | 48        |
| What you need .....                                  | 48        |
| Mission 0: Robot lab rules .....                     | 49        |
| Lab A: Two kinds of programs .....                   | 50        |
| Lab B: One motor + one sensor .....                  | 51        |

|                                                      |    |
|------------------------------------------------------|----|
| Lab C: Arm positions (servo) .....                   | 53 |
| Lab D: Driver station thinking .....                 | 53 |
| Lab E: Move by counting .....                        | 54 |
| Scratch / MakeCode / Python → robot program map..... | 55 |
| Mission ready checklist.....                         | 56 |
| What's next.....                                     | 56 |

# Learning Scratch Programming

---

**Ages:** about 8–16 (younger with help)

**Goal:** Learn block coding by watching, remixing, and building games — the same thinking you will reuse later on circuits and the micro:bit.

Scratch is a free, visual language from MIT. You snap colorful blocks instead of typing code, so you can focus on ideas: loops, decisions, variables, and how a program flows.

---

## Why learn Scratch?

---

### For First Lego League and Spike Prime

The [First Lego League Challenge \(FLLC\)](#) uses the [LEGO Education SPIKE Prime set](#). Teams can code missions with Scratch-style blocks or with Python. For many learners ages 9–14, Scratch (or Scratch-like blocks) is the friendliest way to start.

In a typical 12-week FLLC season (about 2–3 hours per week), coding often takes roughly 30–40% of the time. The rest goes to building, testing, and the innovation project. The innovation project (a larger science-expo style challenge) can contribute nearly half of the points toward winning.

### For parents

Scratch is a block-based language designed for kids (often ages 8–16, though younger and older learners use it too). It is visual — drag and drop blocks instead of typing — so it is a gentle entry into programming.

### Why it is worth considering

- **Problem-solving:** Kids break ideas into steps (a game, a story, an animation). Research such as work in the *Journal of Educational Computing Research* (2020) links Scratch use with gains in computational thinking.
- **Creativity:** Stories, games, and art matter more than memorizing syntax. Many kids stay motivated designing their own versions of games they love.

- **A stepping stone:** Scratch is not Python or JavaScript, but it builds habits you reuse in text languages and in tools like MakeCode on the micro:bit. Early STEM exposure can open doors; the U.S. Bureau of Labor Statistics has projected strong growth in tech jobs.
- **Low barrier, high fun:** Blocks snap together, the editor is free at [scratch.mit.edu](https://scratch.mit.edu), and a large community shares projects. A 2022 Code.org survey found visual tools like Scratch help keep learners engaged longer than text-only coding for many beginners.

### It is not required for every child

- **Not essential:** If a child thrives in sports, music, or another craft, forcing Scratch can backfire. Passion drives learning; coding is one path among many.
- **Limited depth:** Scratch will not teach advanced algorithms or data structures by itself. It is a starter, not a full career curriculum.
- **Screen time:** Balance coding with outdoor play and offline making.

---

## Getting started

---

1. Create an account at [scratch.mit.edu](https://scratch.mit.edu).
2. Open the **Tutorials** section (top right once you are signed in) and try one short built-in tutorial so the editor feels familiar.
3. Work through the video path below. Watch a clip, then **code along** in Scratch.
4. When you finish a project, explore shared projects on Scratch, **remix** one you like, and change something small — that is how you learn to read other people's code.

### How to use this guide

- Stay with **Beginner projects** until a few feel easy.
- Move to **Intermediate**, then **Advanced**, when you want harder challenges.
- Items without a video link are project ideas you can invent yourself (or search for on Scratch / YouTube).

---

## Core skills (watch these first)

---

### Foundations

1. [Getting Started with Scratch](#)
2. [Scratching the Surface: X & Y](#)
3. [Conditional Statements: Make Interactive Projects \(Part 1\) | Tutorial](#)

4. [Conditional Statements: Nesting, Debugging, and Beyond \(Part 2\) | Tutorial](#)
5. [How to Use the Backpack in Scratch | Tutorial](#)

## Intermediate topics

1. [Scratch My Blocks, Part 1: Create Custom Blocks | Tutorial](#)
2. [Scratch My Blocks, Part 2: Tips and Tricks | Tutorial](#)
3. [Scratch My Blocks, Part 3: Custom Blocks with Parameters | Tutorial](#)
4. [Turtle Graphics: Using Pen Blocks in Scratch | Tutorial](#)

## Advanced topics

1. [Advanced Topics: Gravity in Scratch | Tutorial](#)
  2. [Advanced Topics: User-Controlled Scrolling Background | Tutorial](#)
- 

## Beginner projects

---

Code along with each tutorial. Pause often. Change colors, speeds, or sprites after it works.

1. [How to Make an "About Me" Project in Scratch | Tutorial](#)
  2. [How to Make a Jumping Game in Scratch | Tutorial](#)
  3. [How to Make a Clicker Game in Scratch | Tutorial](#)
  4. [How to Make a Pong Game in Scratch | Tutorial](#)
  5. [How to Make a Catch Game in Scratch | Tutorial](#)
  6. [How to Make a Mouse Trail in Scratch | Tutorial](#)
- 

## Intermediate projects

---

1. No-internet dino game: [How To make A DINO Game In Scratch!! SUPER EASY!!](#)
2. Rock–paper–scissors: Human vs. Computer (*build your own, or search Scratch for examples to remix*)
3. Snake game: [How To Make A Snake Game In Scratch 3.0!](#)
4. [How to Make a Scrolling Background in Scratch | Tutorial](#)
5. [How to Make a Story in Scratch | Tutorial](#)
6. [How to Make a Character Designer in Scratch | Tutorial](#)
7. [How to Make a Virtual Pet in Scratch | Tutorial](#)
8. [Pixel Art in Scratch 3 Ways | Tutorial](#)

9. [How to Make a Virtual Town in Scratch | RPG | Tutorial](#)
- 

## Advanced projects

---

1. [How To Make A Calculator In Scratch! | Easy tutorial for kids!](#)
2. [How to make a calculator in scratch](#)
3. Tic-Tac-Toe: Human vs. Computer (*build your own — great practice for grids and win checks*)
4. [How To Make a 1 Player PONG GAME in Scratch 3.0!](#)
5. [How To Make A MINECRAFT Game In Scratch 3.0! \(Part 1\)](#)
6. [How To Make A MINECRAFT Game In Scratch 3.0! \(Part 2\)](#)

More examples and coding challenges will be added here over time.

---

## What's next

---

When Scratch conditionals, loops, and “when X happens” events feel familiar:

1. **Optional:** If you have a LEGO SPIKE Prime set (or care about First Lego League), try the short [SPIKE Prime outline](#) — same coding ideas on a robot, using LEGO’s official lessons.
2. **Core path:** Continue to [Beginner Circuits](#). There you will build the same ideas with wires and components — then connect software and hardware with the [micro:bit V2 guide](#).

# LEGO SPIKE Prime (optional)

---

**Ages:** about 9–14\ **Optional:** Skip this module if you do not have a SPIKE Prime set. The core path goes from [Scratch](#) straight to [Beginner Circuits](#).

**Goal:** Use the same block-coding ideas from Scratch on a real robot — without rewriting LEGO’s lesson library here.

---

## Why try SPIKE after Scratch?

---

Scratch teaches loops, events, and “when X happens.” SPIKE Prime uses **Word blocks** (Scratch-like) to drive motors and read sensors on a hub. If you care about [First Lego League](#) or already own the kit, this is a fun side path. It is **not** required before circuits or the micro:bit.

---

## What you need

---

- [LEGO Education SPIKE Prime set](#)
  - The SPIKE App (or browser support where available) via [spike.legoeducation.com](https://spike.legoeducation.com)
  - A charged hub and a flat space to test a simple driving base
- 

## How to learn (use official lessons)

---

LEGO already publishes step-by-step units. Use those instead of inventing a second curriculum here:

1. Open [spike.legoeducation.com](https://spike.legoeducation.com) or the SPIKE App.
2. Start with beginner / Getting Started units.
3. Code along in **Word blocks**.
4. When a unit feels easy, remix it — change speeds, distances, or sensor reactions the way you remixed Scratch projects.

Python on SPIKE exists later; stay with Word blocks for this outline.

---

## Learning outline (checklist)

---

Before you leave this optional module, you should be able to:

- Connect the hub and run a downloaded program
  - Drive and turn a simple robot with Word blocks
  - Use at least one sensor (color, distance, or force) to change behavior
  - Use loops and conditionals the same way you did in Scratch
  - Complete several official beginner units in the SPIKE App
- 

## Scratch → SPIKE reminder

---

| Scratch idea                     | On SPIKE (Word blocks)     |
|----------------------------------|----------------------------|
| <code>forever</code> / loops     | Repeat / forever loops     |
| <code>if</code> / wait until     | Wait until / if conditions |
| Events ( <code>when ...</code> ) | Events and sensor triggers |
| Sprite on screen                 | Robot (motors + sensors)   |

---

## What's next

---

Continue the **core path** with [Beginner Circuits](#) — wire LEDs, buttons, and sensors yourself on a breadboard. After that comes the [micro:bit V2 guide](#).

# Beginner Circuits

---

Welcome to the hardware lab! Whether you already code in Scratch, build with LEGO Spike Prime, or are just getting started with electronics, this guide helps you learn circuits and components — the foundation for programming real embedded projects.

**Path so far:** If you are following the course in order, you may already know [Scratch](#). The same ideas — loops, “when something happens,” and and/or logic — show up here as wires and parts. After these missions, continue to the [micro:bit V2 guide](#).

In coding, you build with blocks or text. Here, you are going to build with electrons. This guide has **Mission 0 (safety)** plus **8 circuit missions**. For each circuit, you will wire the parts, run an experiment, and observe what happens to discover the hidden rules of electronics.

You’re ready for the full sequence — including transistors, logic, and an oscillator — so treat every circuit as a real mission, not a demo.

Blueprints in this guide use **real electrical schematic symbols**. Learn them once in Mission 0, then read every circuit like a pro.

---

## What you need

---

### Power (needed with either option)

Starter kits do **not** usually include a battery. Also get:

| Part               | Quantity | Notes                                                                            |
|--------------------|----------|----------------------------------------------------------------------------------|
| 9V battery         | 1        | Alkaline is fine                                                                 |
| 9V battery adapter | 1        | Snap connector to <b>DC barrel jack</b> (fits the kit’s breadboard power module) |

Plug the adapter into the power module’s barrel jack, set the module to **5V**, then use the module’s rails for **+5V** and **GND**. Unplug or switch the module off before changing wires.

## Option A: Electronic Starter Kit (easiest)

**Electronic Starter Kit** — a collection of essential parts like resistors, LEDs, and jumper wires.

- [Amazon option 1](#) or [Amazon option 2](#)

These kits usually include a breadboard, 5V power module, jumper wires, LEDs, resistors, buttons, a potentiometer, a photoresistor, capacitors, and NPN transistors.

### Kit notes for this guide

- Many kits ship **2N2222** (or PN2222A) transistors instead of **2N3904**. They work the same electrically, but the **pin order is often different** — see the transistor tip in Circuit 6 before you wire either part.
- Circuit 8 asks for **470Ω** LED resistors. If your kit has **330Ω** instead (common), use those — the flasher still works.

## Option B: Buy parts individually

If you prefer to shop à la carte, get at least this set (covers Circuits 1–8 with a little spare), **plus** the 9V battery and adapter above:

| Part                                   | Quantity | Notes                                         |
|----------------------------------------|----------|-----------------------------------------------|
| Solderless breadboard (830 tie-points) | 1        | Half-size boards also work                    |
| 5V breadboard power supply module      | 1        | Barrel jack for the 9V adapter (USB optional) |
| Male-to-male jumper wires              | 1 pack   | Assorted lengths help                         |
| LEDs (5 mm)                            | 4+       | Red is fine; mix colors if you like           |
| 220Ω resistors                         | 4+       | LED bodyguards for most circuits              |
| 470Ω resistors                         | 2        | Circuit 8 LED protectors (330Ω OK if needed)  |
| 10kΩ resistors                         | 3+       | Transistor base resistors                     |
| Push buttons (breadboard tactile)      | 2        | 4-pin style that fits a breadboard            |
| 10kΩ potentiometer                     | 1        | Panel or trimmer with 3 pins                  |
| Photoresistor (LDR)                    | 1        | Light sensor                                  |
| 2N3904 NPN transistors                 | 2        | Or 2N2222 (different pinout — see Circuit 6)  |
| 100μF electrolytic capacitor           | 1        | Polarized — watch + / –                       |
| 10μF electrolytic capacitors           | 2        | Polarized — watch + / –                       |

## Mission 0: The Maker Rules

---

Before you plug in a single wire, remember these rules:

1. **Power Off First:** Always unplug your battery module before changing wires.
2. **LEDs have a direction:** Current through an LED only works one way. The **long leg (anode)** points toward the higher-voltage / Positive (+) side of that path.
3. **Capacitors are polarized (watch the stripe):** Electrolytic capacitors store charge both ways while charging and discharging, but they must be installed with the correct polarity or they can be damaged. **Long leg / + mark** toward the more positive side; **stripe / short leg** toward the more negative side (often GND).
4. **Resistors are Bodyguards:** An LED connected directly to 5V power will POP and burn out instantly! Always protect your LED by putting a **220Ω** resistor in its path.
5. **Never connect + straight to –:** That makes a short circuit. No “spark tests.”
6. **Stuck? Ask before flipping forever:** If an LED will not light, check the troubleshooting list below. Flip the LED once if needed — then ask for help.

### Quick vocabulary

| Word              | Meaning                                                  |
|-------------------|----------------------------------------------------------|
| <b>Circuit</b>    | A complete loop so electricity can travel out and back   |
| <b>GND (–)</b>    | Ground — the return path for current                     |
| <b>Resistor</b>   | Limits current (protects LEDs and transistor bases)      |
| <b>Series</b>     | Parts in one chain; same current, shared voltage         |
| <b>Parallel</b>   | Separate paths from the same power rails                 |
| <b>Transistor</b> | An electronic switch controlled by a small base signal   |
| <b>Capacitor</b>  | Stores and releases charge over time                     |
| <b>Schematic</b>  | A drawing of a circuit using standard electrical symbols |

### Mission 0.5: Learn the Electrical Symbols

Engineers don’t draw photos of every part — they draw **symbols**. Once you know these, you can read almost any beginner circuit diagram.

## Electrical Symbols Used in This Guide

### Power and Ground

— Wire  
**+5V** Positive supply

⏏ GND (Ground)

### Tactile Push Button

⏏  
 Open until pressed

### NPN Transistor (2N3904)

⏏ C (Collector)  
 B  
 ⏏ E (Emitter to GND)

### Resistor

⏏  
 220 ohm, 10k ohm

### Potentiometer

⏏  
 Variable resistor (knob)

### LED

⏏  
 Triangle to bar

### Photoresistor (LDR)

⏏  
 Changes with light

### Capacitor (polarized)

⏏  
 + = long leg

Tip: LED current goes anode (long leg / triangle) to cathode (short leg / bar)  
 Schematics show connections. Your breadboard is where you plug them in.

## How to read a schematic

1. Find **+5V** (top / start of the path) and **GND** (bottom / end of the path).
2. Follow the wires — a line means a connection.
3. Match each symbol to a real part on your breadboard.
4. Check direction for LEDs and polarized capacitors (arrow / + mark).

| Symbol means...                     | On your breadboard...                  |
|-------------------------------------|----------------------------------------|
| Zigzag resistor                     | Colored bands resistor                 |
| LED triangle + bar                  | LED (long leg = anode / triangle side) |
| Switch gap                          | Tactile push button (4-pin kit button) |
| Pot with arrow                      | Knob (potentiometer)                   |
| Resistor in a circle + light arrows | Photoresistor (LDR)                    |
| Capacitor with +                    | Electrolytic capacitor                 |
| Circle with C/B/E                   | Transistor (2N3904)                    |

## Mission 0.6: Breadboard Basics

A breadboard is a plastic board full of holes that connect inside — like LEGO for wires.

- **Middle area:** Each short row of 5 holes (a–e, and f–j) is connected **side to side**. Parts in the same row touch electrically.
  - **Power rails:** The long strips on the sides (often marked **+** and **–**) run the length of the board. Put **+5V** on the red/+ rail and **GND** on the blue/– rail.
  - **Tip:** Leave a gap (the center ditch) between the left and right halves — they do **not** connect across that ditch.
  - **Schematic ↔ breadboard:** The schematic shows *what connects to what*. The breadboard is *where* you plug those connections.
- 

## If it doesn't work (Troubleshoot)

---

Check these in order:

1. Is power plugged in and turned on?
  2. Is there a complete loop from +5V back to GND?
  3. Is the LED the right way around (long leg toward + / anode toward higher voltage)?
  4. Is there a 220Ω resistor protecting the LED?
  5. Are wires in the same breadboard row when they should touch?
  6. For a tactile push button: does it **straddle the center ditch**, and are you using **opposite corners**? (Those pins connect only when pressed.)
- 

## Phase 1: The Basics of Flow

---

In Scratch, code runs in a loop. In electronics, electricity must complete a continuous circle (a circuit) from the Positive (+) power to the Negative (–) Ground.

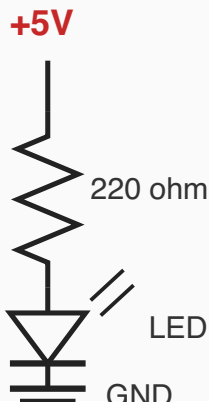
### Circuit 1: The Circle of Power (The Loop)

**The Goal:** Make an LED light up and learn how a circuit breaks.

**Parts Needed:** 5V Power, 1x 220Ω Resistor, 1x Red LED.

**Schematic:**

## Circuit 1 - LED Loop



Same circuit left to right:



**Symbols in this circuit:** +5V · resistor · LED · GND

### The Discovery (Observation)

**Experiment:** Once the light is on, pull one end of the resistor out of the breadboard.

**What happened?** The light turns off instantly.

**The Concept:** Electricity is like a train on a track. If there is a gap anywhere in the track, the train stops completely!

**You built it!** You learned: a circuit must be a closed loop.

---

## Circuit 2: Press for Action (Tactile Push Button)

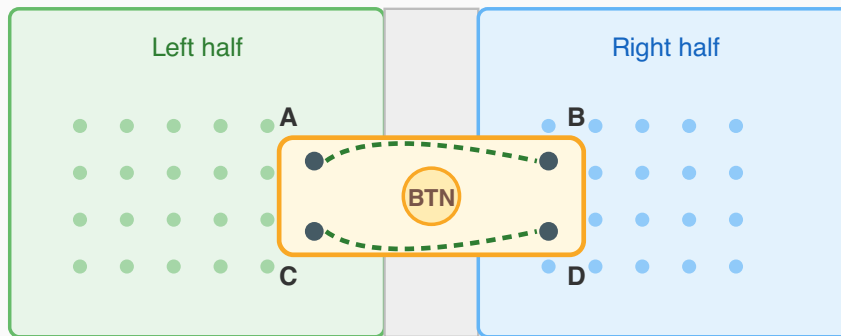
**The Goal:** Use the **4-pin tactile push button** from your starter kit to control an LED — the hardware version of `when space key pressed` in Scratch.

**Parts Needed:** 5V Power, 1x tactile push button (from the kit), 1x 220Ω Resistor, 1x Red LED.

Starter kits usually include several of these little square buttons with four bent legs. They click when pressed and spring back when you let go.

### How to seat the kit button

## Kit tactile button on a breadboard



### Always connected

A to B, and C to D  
(across the ditch)

### Connect when pressed

Use opposite corners:  
A to D, or B to C

1. Seat the button so it straddles the center ditch (two legs each side).
  2. Wire opposite corners into your circuit - those pins join only while you press.
  3. Pins across the ditch on the same row are already joined inside the button.
- Do not use those as your switch - use opposite corners instead.

1. Push the button into the breadboard so it **straddles the center ditch** — two legs on the left half, two on the right.
2. Wire **opposite corners** into the circuit (for example top-left and bottom-right). Those pins connect **only while pressed**.
3. Pins across the ditch on the **same row** are already joined inside the button. That is normal — do not use those as your “switch.”

### Schematic:

#### Circuit 2 - Tactile Button + LED



**Symbols in this circuit:** +5V · switch (tactile button) · resistor · LED · GND

### The Discovery (Observation)

**Experiment:** Press the button. The LED lights up immediately. Let go, and it turns off.

**What happened?** Light on only while your finger is down — just like holding a key in Scratch.

**Challenge:** Move one wire to the *same row across the ditch* instead of an opposite corner. What happens? (The LED may stay on even when you are not pressing — those pins are already joined inside the button.)

**The Concept:** The tactile button is a mechanical drawbridge. When you press it, the bridge lowers, completing the “track” from Circuit 1 so the electrons can flow. When you let go, the spring opens the gap again.

**You built it!** You learned: a tactile switch can open or close a circuit.

---

## Circuit 3: Chain vs. Path (Series and Parallel)

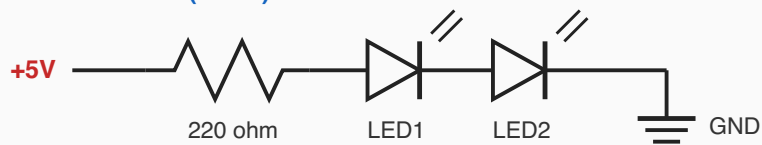
**The Goal:** See what happens when components have to share power.

**Parts Needed:** 5V Power, 2x LEDs, 2x 220Ω Resistors.

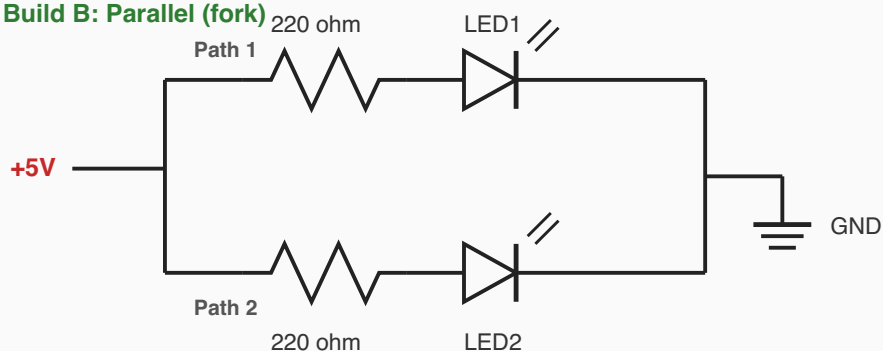
**Schematic:**

### Circuit 3 - Series vs Parallel

#### Build A: Series (chain)



#### Build B: Parallel (fork)



**Symbols in this circuit:** two ways to connect the same parts — **series** (one path) vs **parallel** (forked paths)

## The Discovery (Observation)

**Experiment:** Compare the brightness of the LEDs in Build A vs Build B.

**What happened?** In the Series chain, the lights are very dim (or may barely glow). In the Parallel fork, both are bright.

**The Concept:** In a Series chain, the LEDs have to share the 5V from the power module (same-color LEDs each drop roughly similar voltage). In a Parallel fork, they each get their own path to the full 5V.

**Challenge:** What happens if you try 3 LEDs in series on 5V? (Hint: three LED forward voltages often add up to more than 5V, so they may stay dark — especially blue or white LEDs.)

**You built it!** You learned: series shares; parallel keeps each path strong.

---

## Phase 2: Sensors and Adjustments

### Circuit 4: The Dimmer Knob (Potentiometer)

**The Goal:** Control the flow of electricity smoothly, like a slider variable in Scratch.

**Parts Needed:** 5V Power, 1x 10kΩ Potentiometer (knob), 1x 220Ω resistor, 1x LED.

**Potentiometer tip:** A pot has **3 pins**. Looking at the pins from the front (knob facing you), they are usually **Left — Middle (wiper) — Right**. For this dimmer, we use the pot as a **variable resistor** (rheostat): wire **one outer pin** and the **middle pin** into the circuit. Connect the unused outer pin to the middle pin with a short jumper (recommended), or leave it empty.

**Schematic:**

#### Circuit 4 - Potentiometer Dimmer (rheostat)



**Symbols in this circuit:** potentiometer (zigzag + arrow) · resistor · LED

*(If turning the knob the “wrong way” makes it brighter when you expected dimmer, move the +5V wire to the other outer pin instead.)*

## The Discovery (Observation)

**Experiment:** Slowly turn the potentiometer knob back and forth.

**What happened?** The LED smoothly transitions from very bright to completely off.

**The Concept:** A potentiometer is a variable resistor. Turning the knob is like **pinching a hose** — the more you pinch, the less water (electricity) gets through, so the light gets dimmer!

**You built it!** You learned: resistance can change, and that changes brightness.

---

## Circuit 5: The Sunrise Sensor (Photoresistor)

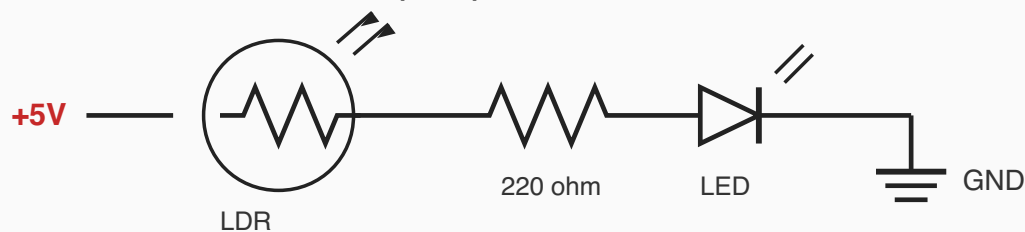
**The Goal:** Make a circuit that reacts to its environment.

**LEGO Analogy:** This is the same kind of idea as the light sensor in your Spike Prime kit!

**Parts Needed:** 5V Power, 1x Photoresistor (LDR), 1x 220Ω resistor, 1x LED.

**Schematic:**

**Circuit 5 - Photoresistor (LDR) Sensor**



**Symbols in this circuit:** LDR (resistor in a circle with light arrows) · resistor · LED

## The Discovery (Observation)

**Experiment:** Cover the photoresistor tightly with your thumb to block the room light.

**What happened?** The LED gets dimmer or turns off when it gets dark.

**The Concept:** The photoresistor changes its resistance based on light. More light = lower resistance (brighter LED). No light = huge resistance (dim/off LED).

**You built it!** You learned: a sensor can change a circuit without a button.

---

## Phase 3: Hardware Magic (Logic & Timers)

---

### Circuit 6: Hardware Logic (The AND Gate)

**The Goal:** Build the Scratch idea of `if <A> and <B>` using physical hardware.

**Pro tip:** Power off, wire carefully, then power on. The 10kΩ base resistor protects the transistor — don't skip it.

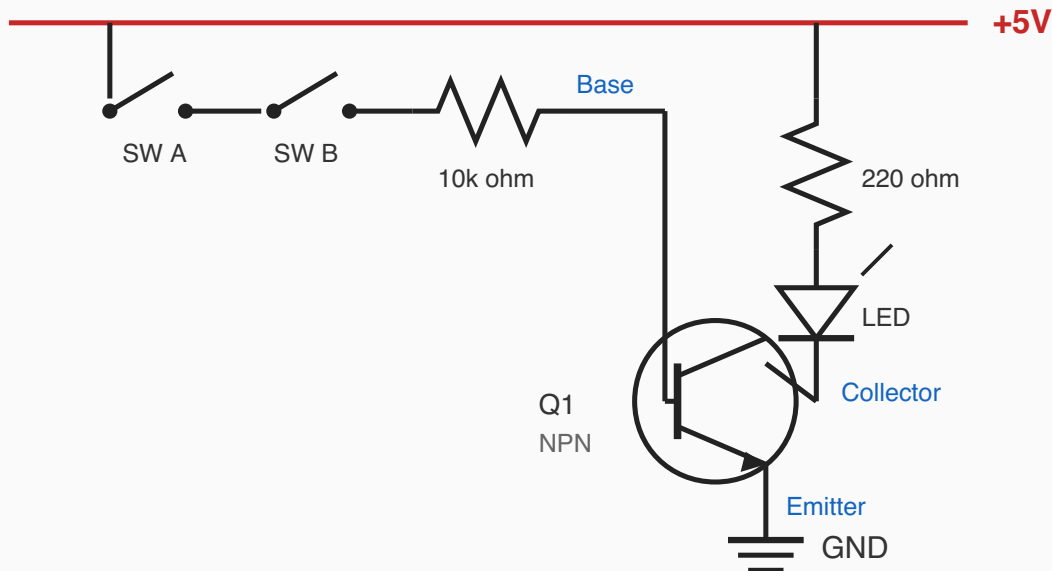
**Parts Needed:** 5V Power, 1x 2N3904 NPN Transistor, 2x Push Buttons, 1x 10kΩ Resistor (base), 1x 220Ω Resistor (LED), 1x LED.

**Transistor tip (pinout matters!):**

- **2N3904:** flat face toward you → pins left to right are usually **E – B – C**.
- **Many kit 2N2222 / PN2222A parts:** flat face toward you → often **C – B – E** (collector and emitter swapped vs 2N3904). Always match the label/datasheet for *your* part — wrong pins mean the circuit will not work.
- In this schematic, **Emitter goes to GND**.

**Schematic:**

## Circuit 6 - Transistor AND Gate



**Symbols in this circuit:** two switches · 10kΩ · NPN transistor · 220Ω · LED

### The Discovery (Observation)

**Experiment:** Press only Switch A. Then press only Switch B. Finally, press both switches at the same time.

**What happened?** The LED only lights up when BOTH are pressed.

**The Concept:** The transistor is an electronic switch. A small signal at the middle pin (the Base) lets a bigger current flow from Collector to Emitter. Both buttons must be pressed to deliver that Base signal — just like Scratch `and`.

**You built it!** You learned: hardware can do logic without writing code.

---

## Circuit 7: The Battery Tank (Capacitor)

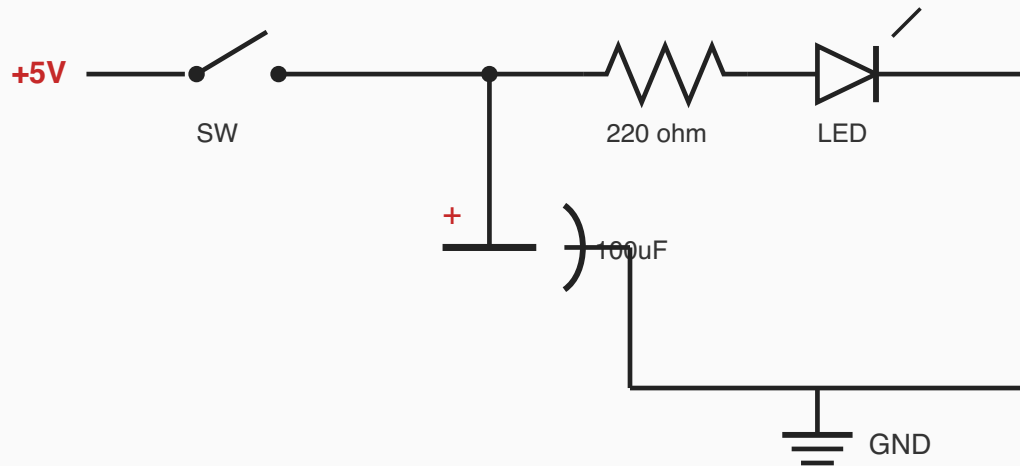
**The Goal:** Store electricity to keep the light on even after the switch is released.

**Parts Needed:** 5V Power, 1x Push Button, 1x 100μF Capacitor, 1x 220Ω Resistor, 1x LED.

(Optional longer fade: if your kit has a larger electrolytic such as  $470\mu\text{F}$  or  $1000\mu\text{F}$ , try it in place of the  $100\mu\text{F}$  — same polarity.)

### Schematic:

#### Circuit 7 - Capacitor Battery Tank



Polarized capacitor: + (long leg) toward switch junction; stripe/short leg toward GND. Fade with  $100\mu\text{F}$  is brief.

**Symbols in this circuit:** switch · polarized capacitor · resistor · LED

After you finish, unplug power. Capacitors can hold a little charge for a moment — that's why the LED does not snap off instantly!

### The Discovery (Observation)

**Experiment:** Hold the button down for 3 seconds. Then let go.

**What happened?** Instead of snapping off instantly, the LED fades for a short moment (with  $100\mu\text{F}$  +  $220\Omega$  it is brief — a blink-out, not several seconds).

**The Concept:** A capacitor is like a water tower. While the button is pressed, it fills up with charge. When you let go of the button, the capacitor empties into the LED path, so the light dies out over a short RC delay instead of cutting off at once.

**You built it!** You learned: capacitors store and release energy.

## Circuit 8: The Flasher (Oscillators)

**The Goal:** Create an automatic timer loop without writing any code!

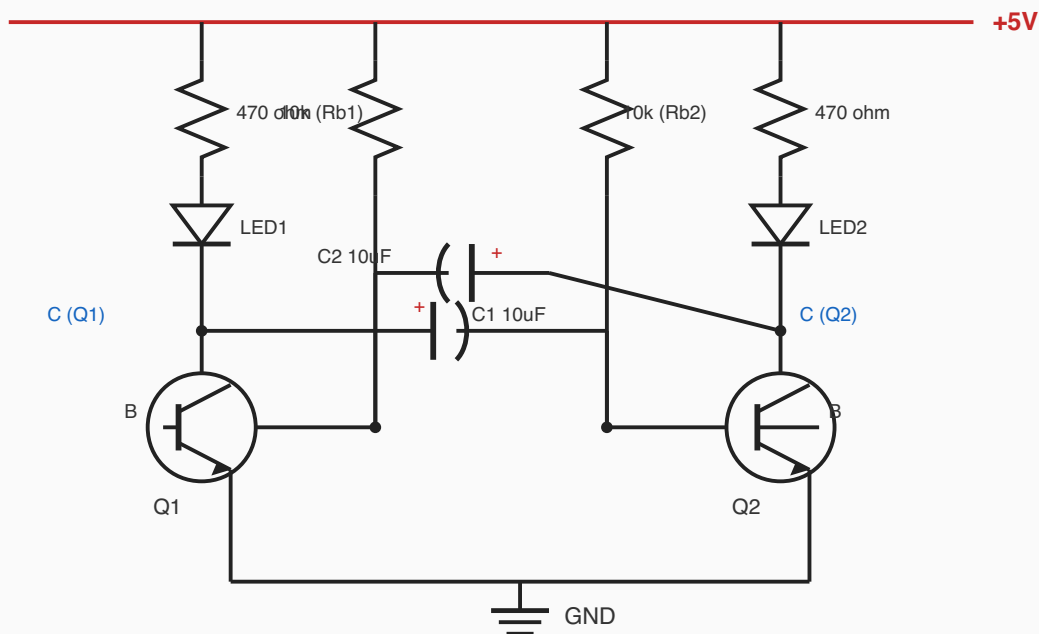
This is the most careful breadboard build in the guide — work slowly, follow the checklist, and double-check polarity on both capacitors.

### Parts Needed:

- 5V Power
- 2x 2N3904 NPN Transistors (Q1, Q2) — or 2N2222 with the correct pinout
- 2x 10 $\mu$ F Capacitors (C1, C2) — watch polarity: + toward the transistor collectors
- 2x LEDs
- 2x 470 $\Omega$  Resistors (R\_LED — LED / collector loads; 330 $\Omega$  OK)
- 2x 10k $\Omega$  Resistors (R\_B — base bias to +5V)

### Schematic:

**Circuit 8 - Astable Multivibrator (Flasher)**



Base resistors go to +5V. Caps couple each collector to the opposite base (no resistor in that path).

**Symbols in this circuit:** two NPN transistors · two polarized capacitors · four resistors · two LEDs

### Build checklist (same as the schematic):

1. Both emitters → GND.
2. Each collector → its own LED → 470 $\Omega$  → +5V.

3. Each base → its own 10kΩ → **+5V** (these bias resistors are required).
4. C1 from Q1 collector **directly** to Q2 base (no resistor in that path).
5. C2 from Q2 collector **directly** to Q1 base (no resistor in that path).
6. Capacitor **+** toward the collector side; stripe toward the base side.

## The Discovery (Observation)

**Experiment:** Wire this up carefully. You do not need to press any buttons. Just connect power and watch.

**What happened?** The LEDs flash back and forth automatically!

**The Concept:** This is an **astable multivibrator** — it has no stable “rest” state, so it keeps flipping. When Q1 turns **on**, its collector voltage falls and C1 pulls Q2’s base down (Q2 off). Then C1 recharges through Q2’s 10kΩ base resistor until Q2 turns on, which pulls Q1 off through C2 — and the cycle repeats. That repeating pulse is a simple hardware “tick-tock” (real computer clocks are more precise, but the idea starts here!).

**You built it!** You learned: circuits can create loops in hardware, not only in code.

---

## What’s next

---

You can now read a schematic, protect an LED, use a switch and sensors, and build simple logic and timing in hardware.

1. Continue to the [Guide to micro:bit V2](#) to connect coding blocks (MakeCode) to a real board with sensors and outputs.
2. After micro:bit, come back for [Intermediate Circuits](#) (same Circuits module) before Arduino — motors, diodes, and stronger switching.
3. Revisit [Learning Scratch Programming](#) anytime you want more practice with the same logic ideas in software.
4. Came from SPIKE? You already moved motors and sensors in a closed kit — here you wire the parts yourself.

# Intermediate Circuits

---

This is the **Intermediate** section of the Circuits module (after [Beginner Circuits](#)).

**Prerequisites:** Finish Beginner Circuits. Completing the [micro:bit V2 guide](#) first is strongly suggested so you already know why boards need sensors, switches, and stronger outputs.

**Goal:** Level up breadboard skills you will need before Arduino — transistors that drive real loads, diodes, voltage dividers, a small DC motor with a flyback diode, and the intuition behind PWM speed control.

This section has **Mission 0 (safety + new symbols)** plus **5 circuit missions**. Same lab style as Beginner: wire it, observe it, name the idea.

---

## What you need

---

Reuse your Beginner Circuits kit (breadboard, 5V module, 9V battery + adapter, jumpers, LEDs, resistors, buttons, potentiometer, NPN transistors). Add:

| Part                                       | Quantity | Notes                                   |
|--------------------------------------------|----------|-----------------------------------------|
| Silicon diode (1N4001 / 1N4007 or similar) | 2+       | Stripe = cathode                        |
| Small DC hobby motor (3–6V)                | 1        | Toy / STEM motor is fine                |
| 1kΩ resistors                              | 2+       | Stronger base resistors for motor drive |
| Multimeter (optional but helpful)          | 1        | For the voltage-divider mission         |

### Power habits for motors

- Keep the 5V breadboard module for these labs if the motor is tiny and spins freely.
  - If the motor stalls, brown-outs the board, or gets hot: stop. Use a separate 3–4×AA pack for the motor later, and **tie all GNDs together**.
  - Never drive a motor by plugging it straight into a micro:bit or Arduino pin.
-

## Mission 0: Intermediate Maker Rules

---

Everything from Beginner Mission 0 still applies. Add these:

1. **Motors kick back:** When a motor stops, it can spit a voltage spike. A **flyback diode** gives that spike a safe path. Do not skip it.
2. **Signal vs power:** A button or board pin should control a transistor. The transistor (and the power rails) should feed the motor or heavy LEDs.
3. **Common ground:** If you ever use two power sources, their GND wires must meet.
4. **Power off before rewiring** — especially around motors and diodes.

### New symbols

#### New Symbols in Intermediate Circuits

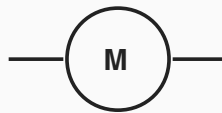
You already know resistors, LEDs, buttons, capacitors, and NPN transistors from Beginner Circuits.

##### Diode (one-way valve)



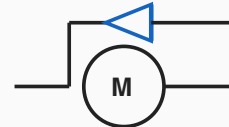
Current flows triangle to bar  
Bar side = cathode (stripe on part)

##### DC motor



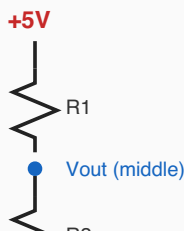
Spins when current flows

##### Flyback diode (on a motor)



Diode across motor (reverse)

##### Voltage divider



##### Power habit reminder

Motors need more current than an LED.

A tiny board pin should only send a signal to a transistor - not feed the motor alone.

Always share GND when mixing power sources.

---

## Phase 1: Stronger Switching

### Circuit 1: One Finger, Two LEDs (Transistor Load)

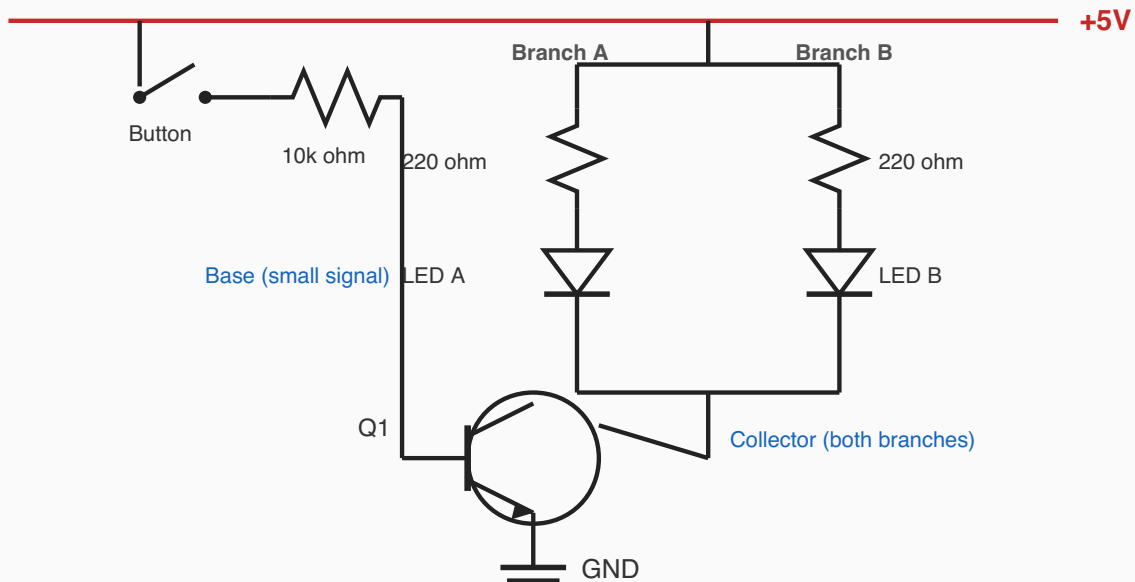
**The Goal:** See that a small base signal can switch a bigger collector path — the same idea boards use to turn on lights and buzzers.

**Parts Needed:** 5V Power, 1× push button, 1× 10kΩ resistor, 1× NPN transistor (2N3904 / 2N2222 — check pinout), 2× 220Ω resistors, 2× LEDs.

**Pinout reminder:** 2N3904 is usually **E–B–C** (flat toward you). Many kit 2N2222 / PN2222A parts are **C–B–E**. Emitter still goes to GND in this circuit.

**Schematic:**

### Circuit 1 - Transistor Switches a Heavier Load



Two parallel LED paths share one transistor switch — more collector current than one LED alone.

**Symbols in this circuit:** button · 10kΩ · NPN · two parallel LED branches · GND

### The Discovery (Observation)

**Experiment:** Press the button. Both LEDs should light. Release — both go dark.

**What happened?** One small control path turned on two parallel LED branches — more collector current than a single LED alone.

**The Concept:** The transistor is still an electronic switch (like Beginner Circuit 6), but now the collector carries a heavier load (two bright branches in parallel). A micro:bit or Arduino pin is like that tiny base signal — it should not try to feed a hungry motor by itself.

**Challenge:** Remove one LED branch so only one LED remains. Does the button still feel the same?

**You built it!** You learned: small signal in, stronger switched load out.

## Circuit 2: The One-Way Valve (Diode)

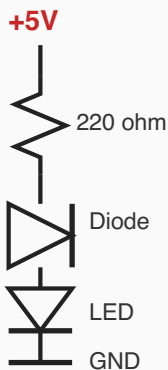
**The Goal:** Prove a diode only lets current through one way — then use that idea to protect motor circuits.

**Parts Needed:** 5V Power, 1× diode, 1× 220Ω resistor, 1× LED.

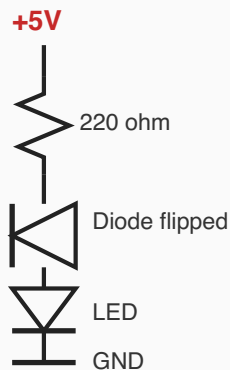
**Schematic:**

### Circuit 2 - Diode Direction Test

#### Build A - diode forward (LED on)



#### Build B - diode reversed (LED off)



**Symbols in this circuit:** resistor · diode · LED

### The Discovery (Observation)

**Experiment:** Build A with the diode stripe (cathode / bar) toward the LED. The LED should light. Power off, flip the diode for Build B — the LED should stay off.

**What happened?** Forward = current flows. Reverse = blocked.

**The Concept:** A diode is a one-way valve. On a motor, we install it *backward across the motor* so normal running current ignores it, but a dangerous kickback spike has a safe loop home.

**You built it!** You learned: diodes enforce direction.

---

## Phase 2: Signals Boards Can Read

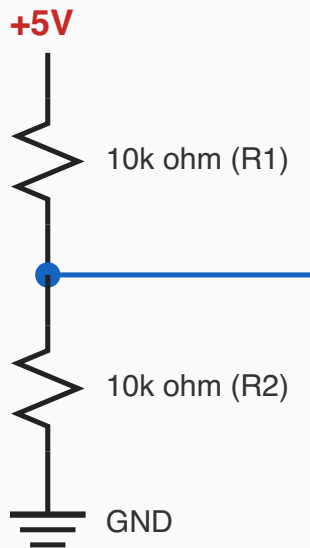
### Circuit 3: Split the Volts (Voltage Divider)

**The Goal:** Make a middle voltage boards can measure — the pattern behind many sensors.

**Parts Needed:** 5V Power, 2× 10kΩ resistors. Optional: multimeter. Optional demo LED + 220Ω from the middle node to GND (may be dim — that is OK).

**Schematic:**

### Circuit 3 - Voltage Divider



#### Equal resistors tip

If  $R1 = R2$ ,  $V_{out}$  is about half of +5V (around 2.5V).

Boards read this middle point as a sensor signal.

**Symbols in this circuit:** two resistors in series · middle tap `Vout`

### The Discovery (Observation)

**Experiment:** With equal 10kΩ resistors, measure (or estimate) the middle point. It should be about half of 5V. Swap the bottom resistor for a photoresistor if you still have one from Beginner Circuits — watch `Vout` change with light.

**What happened?** Two resistors share the supply. The middle voltage depends on both values.

**The Concept:** A voltage divider turns a changing resistance (light, bend, temperature sensor) into a changing voltage. Arduino and the micro:bit read that voltage with an analog input.

**Challenge:** If  $R2$  is much smaller than  $R1$ , is `Vout` closer to GND or to +5V?

**You built it!** You learned: sensors often speak in voltages made by dividers.

## Phase 3: Motors and Speed

### Circuit 4: Motor + Flyback Diode

**The Goal:** Spin a small DC motor with a transistor switch — safely.

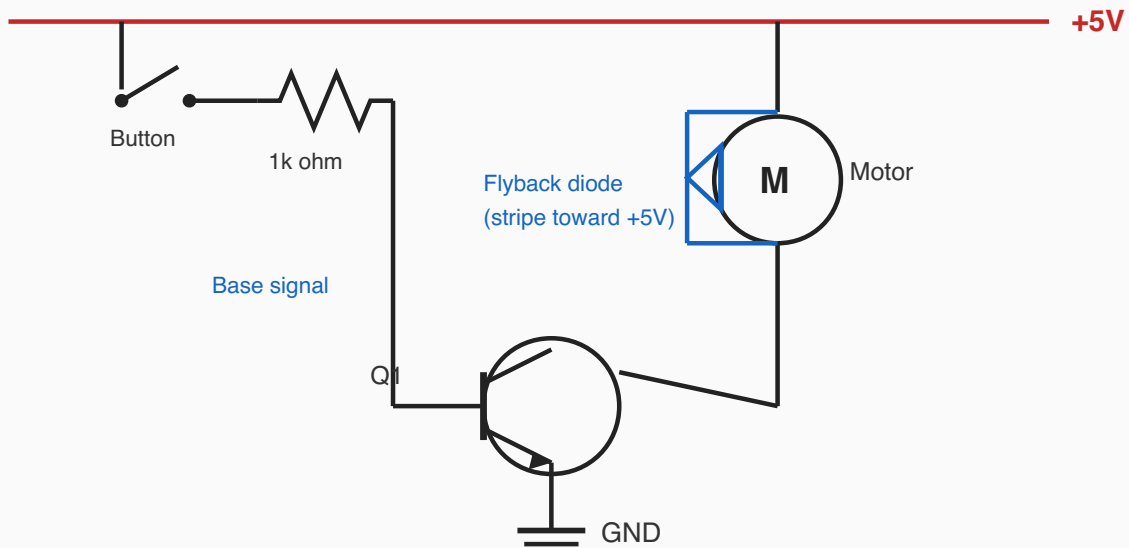
**Parts Needed:** 5V Power, 1× push button, 1× 1kΩ resistor, 1× NPN transistor, 1× small DC motor, 1× diode (flyback).

#### Build tips

- Diode **stripe toward +5V** (cathode to the positive motor terminal).
- Motor may need a firm push to start; do not stall the shaft with your fingers.
- If the 5V module resets or the motor is too hungry, stop and ask for help before adding a second battery pack.

#### Schematic:

##### Circuit 4 - Transistor Motor Driver + Flyback Diode



Power off before wiring. Motor spins one way while the button is pressed.

**Symbols in this circuit:** button · base resistor · NPN · motor · flyback diode

#### The Discovery (Observation)

**Experiment:** Press the button. The motor should spin. Release — it stops.

**What happened?** The transistor carried the motor current. The diode stands ready for kickback when the motor turns off.

**The Concept:** This is the classic low-side switch pattern you will reuse with Arduino: pin → resistor → base; motor between +V and collector; diode across the motor; emitter to GND.

**Challenge:** Remove the diode *only if an adult agrees and you power off between tries* — or better, leave it on and explain to someone why it must stay.

**You built it!** You learned: motors need a driver transistor and a flyback diode.

---

## Circuit 5: The Throttle (PWM Intuition)

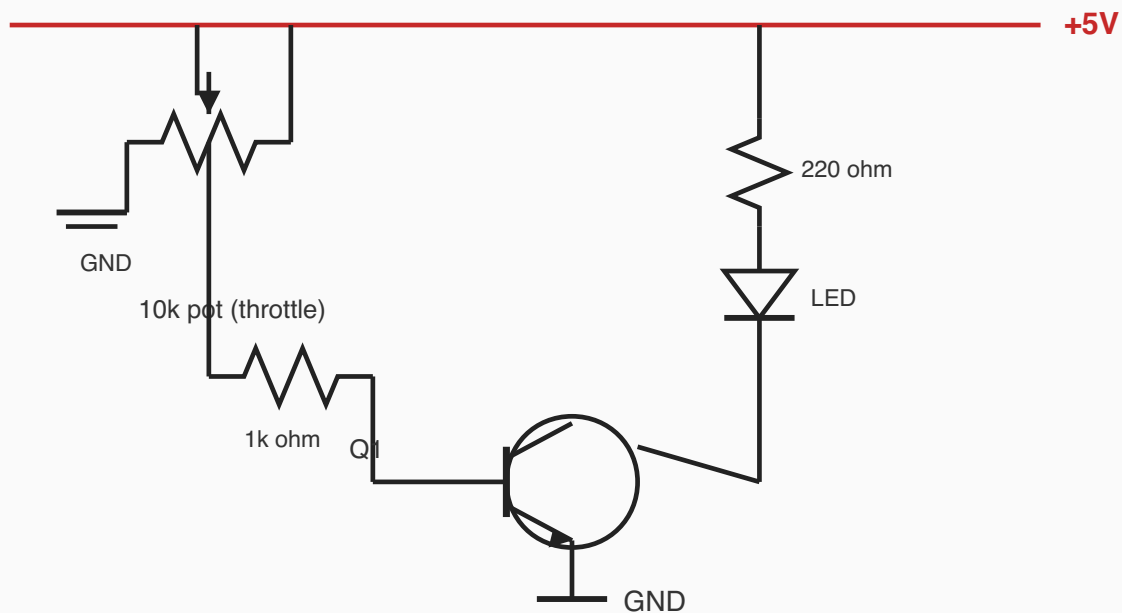
**The Goal:** Dim an LED smoothly with a pot through a transistor — then connect that feeling to PWM on a microcontroller.

**Parts Needed:** 5V Power, 1× 10kΩ potentiometer, 1× 1kΩ resistor, 1× NPN transistor, 1× 220Ω resistor, 1× LED.

*(Optional advanced try: replace the LED path with the motor from Circuit 4, keep the flyback diode, and throttle carefully. Stop if anything gets hot.)*

**Schematic:**

### Circuit 5 - Analog Throttle (PWM Intuition)



Turning the pot changes average brightness - boards do this with fast on/off (PWM).

**Symbols in this circuit:** potentiometer · base resistor · NPN · LED

## The Discovery (Observation)

**Experiment:** Turn the pot slowly. Brightness should change smoothly.

**What happened?** You changed how hard the transistor turns on by changing the base drive — an **analog** throttle.

**The Concept:** Microcontrollers often fake a throttle with **PWM**: switching fully on and fully off very fast. Your eye (or a motor) averages the pulses into a brightness or speed. Same goal as this knob — different method that boards do well.

**You built it!** You learned: analog control today; PWM is the digital cousin on micro:bit and Arduino.

---

## Mission wrap: Power checklist (before Arduino)

---

Before you move on, you should be able to say “yes” to:

- [ ] I can explain why a board pin should not feed a motor directly
  - [ ] I know a diode has a direction (stripe / cathode)
  - [ ] I can sketch a transistor low-side switch with a flyback diode
  - [ ] I know a voltage divider makes a middle voltage for sensors
  - [ ] I remember: common GND when mixing power sources
- 

## What's next

---

You are ready for text-based boards that mix code with real wiring.

1. **Recommended next:** [Guide to CircuitPython](#) (Python on a Pico — gentler text step after Scratch / MakeCode).
2. **Or** [Guide to Arduino](#) (Uno + C++ sketches) if you need that path now. You can do both; CircuitPython first is usually easier.
3. After a text path → [Robot Missions](#) (timed missions and driver control).
4. Revisit the [micro:bit V2 guide](#) to try MakeCode projects that use sensors and outputs with cleaner habits.
5. Invent: combine a divider sensor idea with a transistor output (night light, soft start LED, or a motor that only runs when it is bright).

# Guide to micro:bit V2

---

**Ages:** about 10+ (younger with help)

**Goal:** Get the right gear, set up MakeCode, and start building physical-computing projects on the BBC micro:bit V2.

The micro:bit is a tiny computer you can program to make games, wearables, robots, and sensor projects. This guide helps you choose parts, plug in for the first time, and find tutorials that match your level.

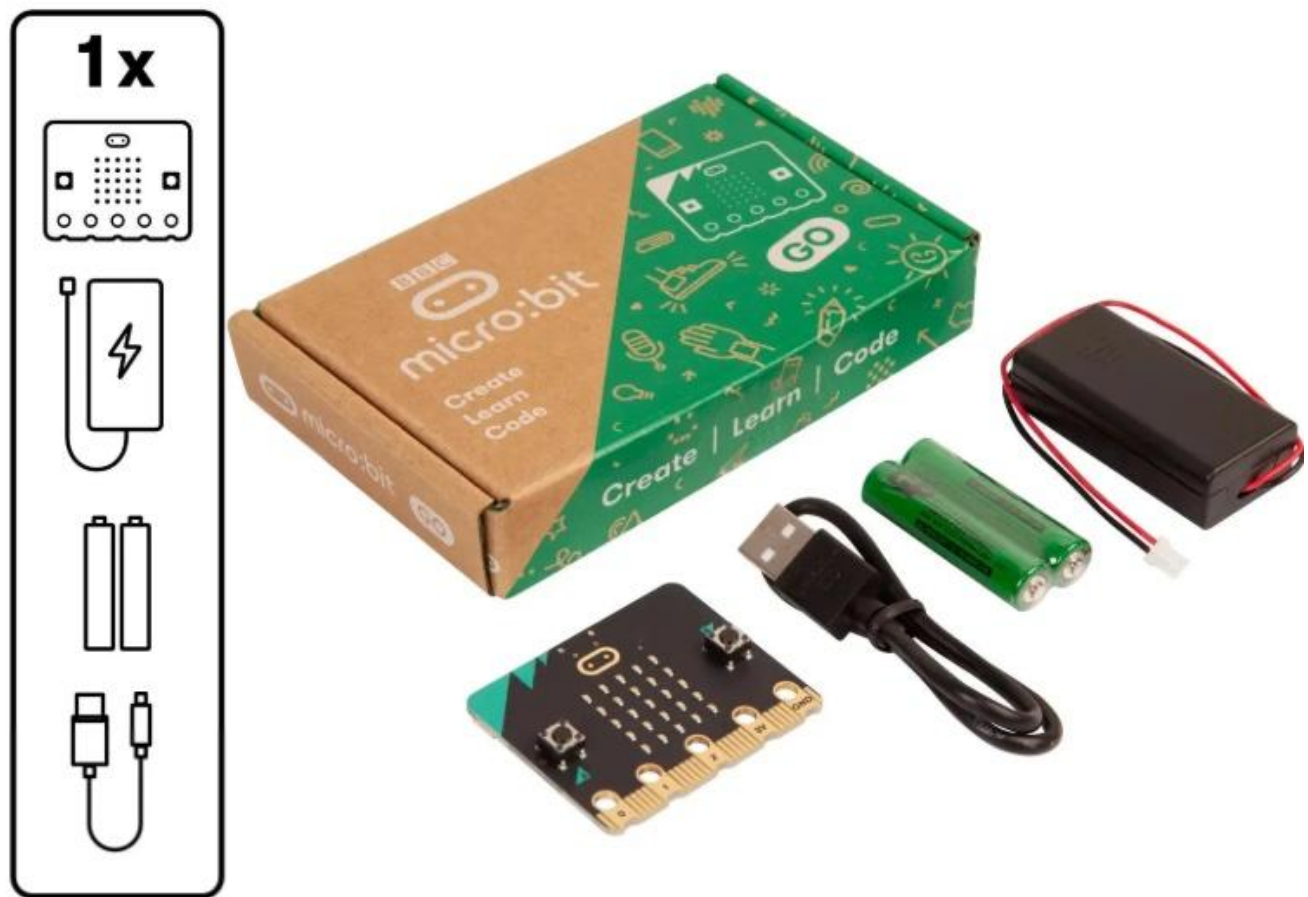
*Disclaimer: There is no affiliation with Amazon; product links are for convenience only.*

---

## 1. Getting your gear

---

Start with the **micro:bit V2 Go Bundle** — it includes the board, a USB cable, and a battery pack.



| Item                                     | What it does                                                      | Where to find it                                                                         |
|------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>micro:bit V2 Go Bundle</b>            | The “brain” of your project, with built-in sensors and a speaker. | <a href="#">Official micro:bit Store</a> , <a href="#">Micro:bit V2 bundle on Amazon</a> |
| <b>Sensor expansion board</b>            | Makes it easier to plug in extra sensors without fiddly wiring.   | <a href="#">Keyestudio KS0360</a> , <a href="#">Buy on Amazon</a>                        |
| <b>Electronic components starter kit</b> | Breadboard parts — resistors, LEDs, jumper wires, and more.       | <a href="#">Electronic Starter Kit</a>                                                   |

*Budget note: the items in this section usually total about US \$50–60.*

## Sensor shield details

The Keyestudio Sensor Shield V2 (KS0360) gives your micro:bit a clearer pin layout for sensors and modules.

1. Documentation: [Keyestudio Sensor Shield V2 for BBC micro:bit](#)
  2. Product page: [Keyestudio KS0360](#)
- 

## 2. Micro:bit vs. Arduino

---

The **micro:bit** is beginner-friendly: LED matrix, buttons, sensors, and a speaker are already on the board. You can learn quickly with almost no extra wiring.

**Arduino** is a versatile platform for more complex electronics. It often needs more setup and external parts to match what the micro:bit includes out of the box. Many learners treat Arduino as a strong **next step** after micro:bit projects feel comfortable.

---

## 3. Extra electronic components

---

For bigger builds (robots, motors, custom sensors — including projects like a “Bat-Bot”-style ultrasonic robot), you will want discrete parts beyond the Go Bundle.

If you are new to breadboards, resistors, and transistors, complete [Beginner Circuits](#) first — then come back here for micro:bit projects that mix board + wiring.

- **Transistors:** [Buy on Amazon](#)
  - **Resistors:** [Buy on Amazon](#)
  - **Diodes:** [Buy on Amazon](#)
  - **Ultrasonic sensor:** Helps a robot “see” distance. [Buy on Amazon](#)
  - **Vibration motors:** Haptic buzz / feedback. [Buy on Amazon](#)
  - **Breadboard:** [Buy on Amazon](#)
  - **9V battery adapters:** [Buy on Amazon](#)
-

## 4. Setting up your lab

---

Program the micro:bit with **Microsoft MakeCode**. It uses colorful blocks — very similar to [Scratch](#) — that snap together so typing mistakes do not block you.

If you want more Scratch practice first, use the [Learning Scratch Programming](#) guide.

You can also switch to **Python** or **JavaScript** in MakeCode when you are ready for text.

1. Go to [makecode.microbit.org](https://makecode.microbit.org).
  2. Click **New Project**.
  3. Connect your micro:bit to the computer with the USB cable.
  4. Download your program to the board and try the on-board LEDs or buttons.
- 

## 5. Video tutorials and project ideas

---

Learning by watching — then rebuilding — is one of the fastest ways to improve. Work top to bottom, or jump to a topic you need.

### Basics of electronic parts

These videos overlap with ideas in [Beginner Circuits](#). Use them as a refresher, or watch before wiring sensors to the micro:bit.

- [How to Use a Breadboard](#)
- [Basic Electronics for Beginners in 15 Steps](#)
- [If You Understand This, You Understand Electronics](#)
- [How Resistor Work - Unravel the Mysteries of How Resistors Work!](#)
- [Fun with Transistors](#)

### Micro:bit V2 tutorials

- Introduction to micro:bit V2
- [Get to know the BBC micro:bit's features](#)
- [How to use a micro:bit | Tutorial #1](#)
- [How to use buttons on a micro:bit | Tutorial #2](#)
- [If/else statements with micro:bit | Tutorial #3](#)
- [Motion control with micro:bit | Tutorial #4](#)
- [Coding with micro:bit - Part 3A - LED lighting](#)
- [Coding with micro:bit - Part 4 - Making Music](#)

- [Coding with micro:bit - Part 5 - Motors & Servos](#)
- [Coding with micro:bit - Part 6 - Coding tips - Variables and Conditionals](#)
- [Coding with micro:bit - Part 7 - Making a Virtual Pet](#)
- [Coding with micro:bit - Part 8 - Making a servo powered waving arm and inch worm](#)
- [Control rotation and speed with Micro:bit](#)
- **Full series start:** [Coding with micro:bit - Part 1 - Introduction](#)

## Projects

- [Play Scratch Games with microbit #stemeducation](#)
  - [Blink LED Using Microbit](#)
  - [3 Flashing LED Lights on a Micro:bit](#)
  - [Micro:bit Solar Tracker I Science Project](#)
- 

## 6. Other starter kits

---

Robot and sensor kits can speed up learning if you want a packaged set of parts:

- [KEYESTUDIO Sensors Box Starter Kit for Micro:bit](#)
  - [KEYESTUDIO Micro:bit V2 Robot Starter Kit \(without Microbit\) for Makecode and Python](#)
- 

## What's next

---

Beginner Circuits should come **before** this guide. If you skipped it:

- New to electronics wiring? → [Beginner Circuits](#)

Otherwise continue the **core path**:

1. [Intermediate Circuits](#) — the second section of the Circuits module (transistors that drive real loads, diodes, voltage dividers, and a safe DC motor switch). **Required before CircuitPython or Arduino.**
2. After Intermediate → [CircuitPython \(recommended\)](#) and/or [Arduino sketches](#).
3. Want stronger block-coding habits? → [Learning Scratch Programming](#)
4. Ready to invent on the micro:bit? Combine MakeCode with sensors, motors, and the expansion board — then share what you build.

# Guide to CircuitPython

---

**Ages:** about 11+ (younger with help)

**Prerequisites (required):**

- [Beginner Circuits](#)
- [Intermediate Circuits](#) — transistors, diodes, voltage dividers, and safe motor switching

**Strongly recommended:** the [micro:bit V2 guide](#) so you already practiced sensors and outputs with blocks.

**Goal:** Program a real board with **Python** (CircuitPython) — a gentler text step after Scratch / MakeCode than jumping straight to Arduino C++ sketches.

**Path choice after Intermediate Circuits**

| Path                                  | Language               | Best when...                                                 |
|---------------------------------------|------------------------|--------------------------------------------------------------|
| <b>This guide (recommended first)</b> | CircuitPython (Python) | You want text code that still reads a bit like Scratch ideas |
| <a href="#">Guide to Arduino</a>      | C++ sketches           | You specifically need Uno / Arduino IDE sketches             |

You can do **both**. Many learners do CircuitPython first, then Arduino later.

*Disclaimer: There is no affiliation with Amazon; product links are for convenience only.*

---

## 1. Getting your gear

---

**Default board:** [Raspberry Pi Pico](#) (or **Pico W** if you want Wi-Fi later). It is inexpensive, widely used with CircuitPython, and easy to find.

**Arduino-branded option:** [Arduino Nano RP2040 Connect](#) — same RP2040 family ideas, Arduino shape and store ecosystem. Setup steps are similar (install CircuitPython UF2 for that board); pin names differ, so follow the board's pinout.

| Item                                  | What it does                      | Where to find it                                              |
|---------------------------------------|-----------------------------------|---------------------------------------------------------------|
| <b>Raspberry Pi Pico</b> (or Pico W)  | The board you program             | <a href="#">Raspberry Pi</a> , <a href="#">Pico on Amazon</a> |
| <b>USB micro cable</b> (data-capable) | Power + copy files / serial       | Often sold with the Pico                                      |
| <b>Breadboard + jumpers + parts</b>   | LEDs, resistors, buttons, sensors | Reuse your <a href="#">Circuits</a> kit                       |
| <b>Optional: Nano RP2040 Connect</b>  | Arduino-shaped RP2040 board       | <a href="#">Arduino Store</a>                                 |

Classic **Arduino Uno** does **not** run CircuitPython. For Uno, use the [Arduino sketch guide](#) instead.

---

## 2. Why CircuitPython after Scratch / MakeCode?

---

- You type **Python**, not C++ — indentation and plain words ( `if` , `while` , `print` ) feel closer to reading Scratch scripts than Arduino sketches do.
- The board shows up like a **USB drive**. You edit `code.py` , save, and it runs — no separate “compile and upload” dance at first.
- Adafruit’s Learn guides are written for beginners and match breadboard projects.

Arduino C++ sketches are still useful (especially for Uno). They are just a bigger jump; save them for when Python on the Pico feels comfortable — or when a project needs Uno.

---

## 3. Install CircuitPython (Pico)

---

Follow Adafruit’s current Pico guide (UF2 files change over time — use the live page):

1. [CircuitPython on Raspberry Pi Pico](#)
2. Put the Pico in bootloader mode (hold **BOOTSEL**, plug in USB, release).
3. Drag the CircuitPython `.uf2` onto the `RPI-RP2` drive.
4. After it reboots, you should see a **CIRCUITPY** drive with `code.py` .

**Editor:** [Mu Editor](#) (CircuitPython mode) or [Thonny](#) are both fine for kids.

**Nano RP2040 Connect:** use Adafruit’s CircuitPython installer page for that board (search “CircuitPython Arduino Nano RP2040 Connect” on [circuitpython.org](#)) and the same Mu/Thonny workflow.

---

## 4. Scratch / MakeCode → Python map

---

| Scratch / MakeCode idea    | CircuitPython                                                        |
|----------------------------|----------------------------------------------------------------------|
| When the program starts    | Top of <code>code.py</code> (runs from the start)                    |
| <code>forever</code>       | <code>while True:</code>                                             |
| Wait / pause               | <code>time.sleep(seconds)</code>                                     |
| Digital write (LED on/off) | <code>led.value = True / False</code> (or <code>digitalio</code> )   |
| Read button                | <code>button.value</code> with <code>digitalio</code> + pull-up/down |
| Analog sensor              | <code>analogio.AnalogIn</code>                                       |
| Print / debug              | <code>print(...)</code> (Serial / Mu plotter)                        |

Blink-style idea on the Pico (onboard LED — exact pin helper may vary by board; see the Adafruit Pico guide for the recommended snippet):

```
import time
import board
import digitalio

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

Save as `code.py` on **CIRCUITPY**. The LED should blink. Change the `0.5` values to remix the timing — same habit as remixing Scratch.

---

## 5. Wiring habits (same as Intermediate)

---

- LED + **220Ω** resistor
- Pin = **signal**, not motor power — transistor + flyback diode for motors
- Common **GND** with any second power source
- Unplug or power off before big rewires

Refresh: [Intermediate Circuits](#).

---

## 6. Curated projects

---

Work through Adafruit Learn (keep the browser guide open; copy patterns into `code.py`):

1. Finish the Pico “getting started” blink / REPL steps on [Getting Started with Pico + CircuitPython](#)
2. External LED on a breadboard — same wiring as Beginner Circuit 1, driven from a GPIO pin
3. Button input — digital read + pull resistor (or internal pull)
4. Potentiometer or light sensor — analog input (voltage divider ideas from Intermediate)
5. Browse more: [CircuitPython Essentials](#) and project guides on [learn.adafruit.com](#)

Remix rule: change pin numbers, delays, or thresholds after each tutorial works.

---

## 7. Learning checklist

---

- ☐ Install CircuitPython so **CIRCUITPY** appears
  - ☐ Edit and save `code.py` so the board runs your program
  - ☐ Explain `while True:` like Scratch `forever`
  - ☐ Blink an **external** LED with a resistor
  - ☐ Read a button or analog sensor
  - ☐ Explain why a motor still needs a transistor driver
- 

## What's next

---

1. **Invent and share** — sensor in, LED / sound / transistor-driven motor out.
2. Continue to [Robot Missions](#) — timed missions and driver control (classroom board or competition robot brain).
3. Ready for Uno + C++ sketches? → [Guide to Arduino](#) (you can do Arduino before or after Robot Missions).
4. Revisit [Circuits](#) or the [micro:bit guide](#) anytime.

# Guide to Arduino

---

**Ages:** about 11+ (younger with help)

**Prerequisites (required):**

- [Beginner Circuits](#)
- [Intermediate Circuits](#) — transistors, diodes, voltage dividers, and safe motor switching

**Strongly recommended:** the [micro:bit V2 guide](#) so you already practiced sensors and outputs with MakeCode.

**Gentler text path:** If C++ sketches feel like a big jump, start with [CircuitPython \(Pico\)](#) first — Python on the board — then come back here when you want Uno + Arduino IDE.

**Goal:** Install the Arduino IDE, upload your first sketch, map Scratch / MakeCode ideas to text code, and follow curated beginner projects — reusing the wiring habits from Circuits.

*Disclaimer: There is no affiliation with Amazon; product links are for convenience only.*

---

## 1. Getting your gear

---

Start with an **Arduino Uno R3** (or a compatible Uno-shaped board). Uno R4 or a Nano also works if you pick the matching board name in the IDE.

| Item                                      | What it does                     | Where to find it                                                 |
|-------------------------------------------|----------------------------------|------------------------------------------------------------------|
| <b>Arduino Uno R3</b> (or compatible)     | The board you program            | <a href="#">Arduino Store</a> , <a href="#">Uno R3 on Amazon</a> |
| <b>USB cable</b> (A to B for classic Uno) | Power + upload                   | Often included with the board                                    |
| <b>Breadboard + jumper wires + parts</b>  | LEDs, resistors, buttons, motors | Reuse your <a href="#">Circuits</a> kit                          |

Optional: an electronics starter kit that already includes an Uno-compatible board and sensors — useful if you want one box, but not required if you already have Circuits parts.

---

## 2. Arduino vs. micro:bit

---

|              | micro:bit                                          | Arduino Uno                                              |
|--------------|----------------------------------------------------|----------------------------------------------------------|
| Built-ins    | LED matrix, buttons, sensors, speaker on the board | Usually just an onboard LED — you wire the rest          |
| Coding start | MakeCode blocks (Scratch-like)                     | Text sketches ( <code>setup</code> / <code>loop</code> ) |
| Strength     | Fast first wins with almost no wiring              | More pins and flexibility for custom circuits            |
| Path here    | Great after Beginner Circuits                      | Next step after Intermediate Circuits                    |

Arduino is not “harder electronics magic” — it is the same breadboard skills with a board that expects you to write (or remix) short text programs.

---

## 3. Install and first upload

---

1. Download the Arduino IDE from [arduino.cc/en/software](https://arduino.cc/en/software) (Desktop IDE). The [Arduino Cloud / Web Editor](#) is a fine alternate if you prefer the browser.
2. Install, open the IDE, and connect the Uno with USB.
3. Choose your board and port: **Tools** → **Board** (e.g. Arduino Uno) and **Tools** → **Port**.
4. Open the Blink example: **File** → **Examples** → **01.Basics** → **Blink**.
5. Click **Upload**. The onboard LED should blink.

Official help:

- [Getting Started with the Arduino Desktop IDE](#)
- [How to write a Sketch](#) — structure of a sketch, `setup()` , `loop()` , and how programs run
- [Built-in Examples](#)

If upload fails: check the cable (some charge-only cables do not work), the Port menu, and that the board selection matches what you plugged in.

---

## 4. Scratch / MakeCode → sketch map

---

Read [How to write a Sketch](#) for the full official guide. The table below is a quick map from Scratch / MakeCode ideas.

| Scratch / MakeCode idea             | Arduino sketch                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------|
| When the program starts             | <code>setup()</code> — runs once                                                   |
| <code>forever</code> / forever loop | <code>loop()</code> — runs again and again                                         |
| Wait / pause                        | <code>delay(millisecods)</code>                                                    |
| Set pin / digital write             | <code>digitalWrite(pin, HIGH)</code> or <code>LOW</code>                           |
| Read button / digital read          | <code>digitalRead(pin)</code>                                                      |
| Read sensor (0–1023 style)          | <code>analogRead(pin)</code>                                                       |
| Dim LED / motor speed idea          | <code>analogWrite(pin, 0–255)</code> on PWM pins (PWM intuition from Intermediate) |
| Variable                            | <code>int</code> , <code>bool</code> , etc. at the top or inside functions         |

A tiny Blink-style sketch (same idea as the built-in example):

```
// setup runs once when the board powers on or resets
void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // onboard LED pin as output
}

// loop runs forever — like Scratch "forever"
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // LED on
  delay(1000);                     // wait 1 second
  digitalWrite(LED_BUILTIN, LOW);  // LED off
  delay(1000);
}
```

Read the comments in IDE examples — they are written for learners.

## 5. Wiring habits (do not skip)

Before you follow project tutorials, keep Intermediate Circuits habits:

- An LED needs a series resistor (often **220Ω**).
- A board pin is a **signal**, not a motor power supply — use a transistor (and flyback diode) for motors.
- Share **GND** whenever the Arduino and another power source are in the same circuit.
- Power off or unplug USB before major rewires.

Refresh: [Intermediate Circuits](#).

---

## 6. Curated projects (build these next)

---

Work top to bottom. Prefer the examples inside the IDE (**File** → **Examples**) and the official docs — then remix (change pins, delays, or thresholds).

### Built-in path (in the IDE)

1. **Blink** — `Examples` → `01.Basics` → `Blink` (onboard LED)
2. **DigitalReadSerial** — read a pin and watch the Serial Monitor
3. **Button** — `Examples` → `02.Digital` → `Button` (external button + LED)
4. **AnalogReadSerial / AnalogInput** — potentiometer or sensor voltage
5. **Fade** — `Examples` → `03.Analog` → `Fade` (PWM brightness)

Docs index: [Built-in Examples](#).

### Wiring + code (after Blink feels easy)

- [How to Wire and Program a Button](#)
- External LED on a breadboard — many kits walk through pin 13 or another digital pin with a 220Ω resistor to GND (same idea as Beginner Circuit 1, plus `digitalWrite`)
- Motor with transistor + flyback diode — reuse [Intermediate Circuit 4](#) and drive the transistor base from an Arduino pin through a resistor (never power the motor from the pin alone)

### Stretch

Browse [Arduino Project Hub](#) for ideas once the built-in examples feel comfortable. Remix one project instead of copying five.

---

## 7. Learning checklist

---

Before you call this module done, you should be able to:

- [ ] Install the IDE (or Web Editor) and upload Blink
- [ ] Explain `setup()` vs `loop()` in your own words
- [ ] Blink an **external** LED with a resistor on the breadboard
- [ ] Read a button or potentiometer in a sketch

- [ ] Explain why a motor needs a transistor driver and flyback diode
  - [ ] Complete several built-in Examples and change something in each
- 

## What's next

---

You reached Arduino sketches on the core electronics path. Many learners also complete [CircuitPython](#) (before or after this guide).

1. Continue to [Robot Missions](#) — timed missions and driver control (classroom board or competition robot brain).
2. **Invent and share** — combine a sensor (divider / pot / LDR) with an output (LED, buzzer, or transistor-driven motor).
3. Prefer Python on a Pico? → [Guide to CircuitPython](#).
4. Revisit [Beginner](#) or [Intermediate Circuits](#) anytime wiring feels shaky.
5. Revisit the [micro:bit guide](#) if you want more block-based practice.

# Robot Missions

---

**Ages:** about 12+ (younger with a mentor)

**Prerequisites (required):**

- [Beginner Circuits](#)
- [Intermediate Circuits](#)
- [CircuitPython](#) or [Arduino](#)

**Helpful first:** the [micro:bit V2 guide](#) (sensors + outputs with blocks).

**Goal:** Build the habits real robot teams use — timed missions, driver control, named parts, safe motors, and move-by-counting — on a classroom board, and on a competition robot brain when your group has one.

---

## You're building mission robots

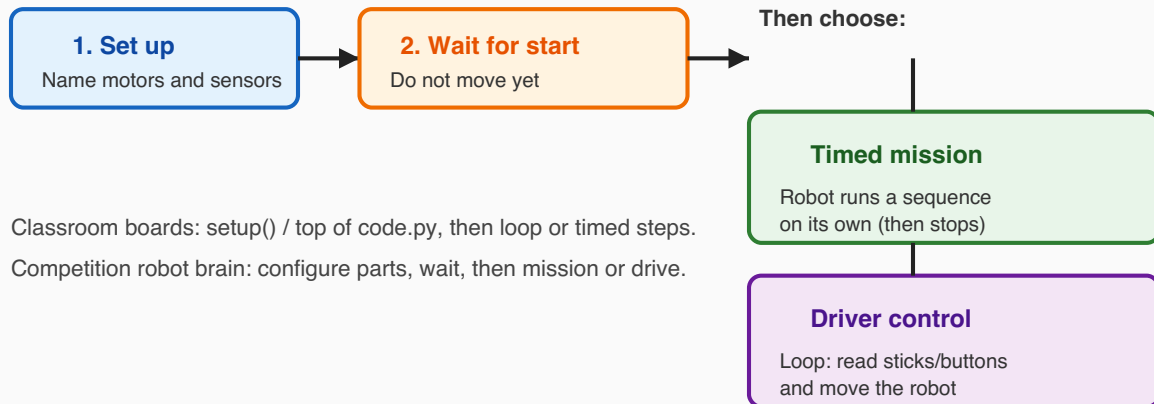
---

A mission robot needs two kinds of programs:

1. **Timed mission** — the robot runs a sequence on its own (drive, stop, move an arm).
2. **Driver control** — a person steers with sticks or buttons while a loop reads the controls and moves motors.

You already practiced loops, sensors, and transistor motor drivers. This module connects those ideas into **robot programs** with a clear start sequence.

## Two kinds of robot programs



## Two ways to practice

| Path                     | What you use                                                             | When                                                         |
|--------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Classroom board</b>   | Pico (CircuitPython) or Arduino Uno + your Circuits kit                  | Always available; finish every lab this way if needed        |
| <b>Competition brain</b> | REV Control Hub (or Expansion Hub), robot battery, motors/servos/sensors | When your team or club has the kit — same labs, hub software |

You can complete the whole module on a classroom board. The competition-brain steps are the same missions with hub motor ports and Blocks (then a little typed code).

## What you need

### Classroom board (Practice path)

Reuse gear from earlier modules:

- Pico with CircuitPython **or** Arduino Uno
- Breadboard, jumpers, 5V power module, 9V battery + adapter
- Push button, potentiometer, LED + 220Ω
- Small DC motor, NPN transistor, 1kΩ base resistor, flyback diode (Intermediate Circuit 4)

- Optional: photoresistor; hobby servo if you have one

## Competition brain (Team kit path)

Short list (mentor / team kit):

- REV Control Hub (or Expansion Hub + compatible phone / Driver Station setup)
- Charged robot battery and safe charging habits
- 1–2 DC motors (encoders preferred for Lab E)
- 1 servo
- 1 touch sensor or color sensor
- USB cable / Wi-Fi as your hub docs require

Follow the current Control Hub getting-started docs from REV for Driver Station pairing and Blocks. Do not invent a second wiring standard here.

---

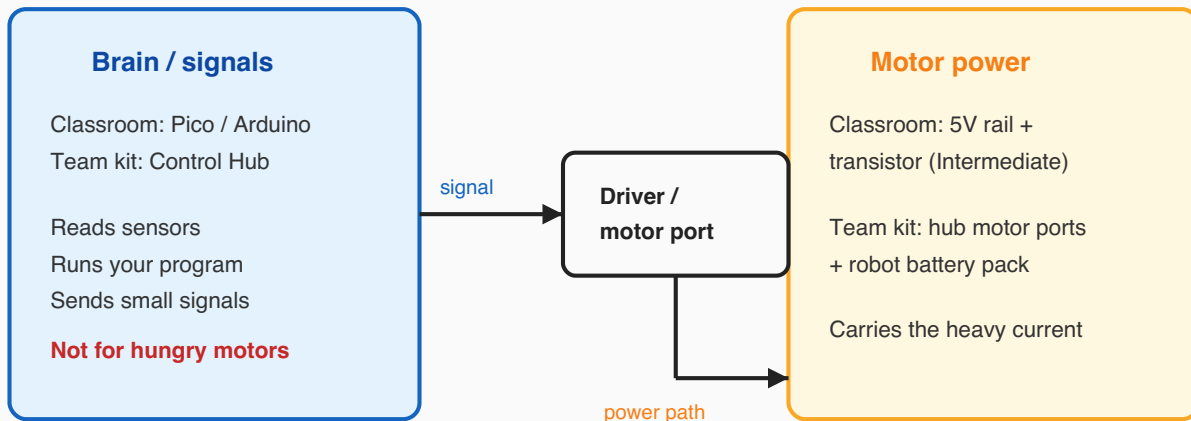
## Mission 0: Robot lab rules

---

Everything from Intermediate Maker Rules still applies. Add these:

1. **Brain ≠ motor power.** A GPIO pin or logic signal must not feed a hungry motor. Classroom path: transistor + flyback diode. Team kit: hub **motor ports** and the robot battery.
2. **Strong batteries deserve respect.** Power off before rewiring. No bare metal across battery terminals. Mentors handle first-time battery / hub setup.
3. **Name every part before you code.** Write a parts list: `left_motor`, `arm_servo`, `bumper` — then wire and configure those names.
4. **Wait for start.** Missions should set up quietly, then move only after a clear start signal (button, mentor “go,” or hub wait-for-start).

## Robot brain vs motor power



Same Intermediate rule: pin = signal; transistor or hub port feeds the motor.

## Lab A: Two kinds of programs

**The Goal:** Feel the difference between a timed mission and a driver-control loop.

### Practice path (classroom board)

**Parts:** Board + 1× LED + 220Ω (and optionally a button).

1. **Timed mission:** Turn the LED on for 1 second, off for 1 second, repeat exactly three times, then stop (exit the loop or sit idle).
2. **Driver control:** In a forever loop, turn the LED on only while a button is held (or toggle with two keys in the Serial monitor if you prefer).

CircuitPython-shaped timed mission:

```

import time
import board
import digitalio

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

# Set up done – wait for a mentor "go", then run the mission
input("Press Enter to start the mission...")

for _ in range(3):
    led.value = True
    time.sleep(1)
    led.value = False
    time.sleep(1)

# Mission finished – do not keep blinking
while True:
    time.sleep(1)

```

Arduino-shaped driver control uses `loop()` to read a button and `digitalWrite` the LED — same idea as Scratch `forever` + `if`.

## Team kit path

1. Create a **Blocks** program on the Control Hub.
2. In setup / init: configure nothing that moves yet (or set motor power to 0).
3. Use **wait for start**.
4. Timed mission: run a motor slowly for 1 second, stop, done.
5. Second program: driver control — forever loop reading a gamepad button to run/stop a motor.

## The Discovery

**What happened?** The timed mission does the same thing every run. Driver control waits on you.

**You built it!** You learned: set up → wait for start → mission **or** drive loop.

## Lab B: One motor + one sensor

**The Goal:** Motor moves only when a sensor says so — named parts, safe drive.

## Practice path

Reuse [Intermediate Circuit 4](#) (motor + transistor + flyback diode). Drive the transistor base from a board pin through the base resistor. Use a push button (or LDR + threshold) as the sensor.

**Parts list example:** `drive_motor`, `bumper` (button).

Behavior: while bumper is pressed, motor runs slowly; release → stop. Never connect the motor straight to a pin.

## Team kit path

1. Plug a DC motor into a hub motor port. Plug a touch (or color) sensor into a sensor port.
2. In Blocks (or Device config): name them `drive_motor` and `bumper`.
3. After wait-for-start: while `bumper` is pressed, set `drive_motor` to a low power; otherwise 0.

## Typed team code bridge (same robot)

When Blocks feels easy, remake **only this lab** in typed hub code. The shape looks like:

```
// Names must match your robot configuration
DcMotor driveMotor;
DigitalChannel bumper;

@Override
public void init() {
    driveMotor = hardwareMap.get(DcMotor.class, "drive_motor");
    bumper = hardwareMap.get(DigitalChannel.class, "bumper");
    bumper.setMode(DigitalChannel.Mode.INPUT);
}

@Override
public void loop() {
    if (bumper.getState()) {
        driveMotor.setPower(0.3);
    } else {
        driveMotor.setPower(0);
    }
}
```

Your mentor's sample OpMode template may use slightly different base classes — match their template; keep the **parts list** → **hardwareMap** → **loop** idea.

## The Discovery

**What happened?** The sensor gates the motor. Names in code match names on the robot.

**You built it!** You learned: named parts list + safe motor drive + sensor gate.

---

## Lab C: Arm positions (servo)

---

**The Goal:** Move an “arm” to three clear positions.

### Practice path

- **If you have a hobby servo:** follow a Pico/Arduino servo tutorial; send three angles (for example 0° / 90° / 180°) with pauses. Use a separate 5V supply for the servo if the board brown-outs; **tie GNDs together**.
- **If you do not:** simulate positions with an LED brightness or three blink patterns named `STOW`, `REACH`, `DROP`. The idea is **named states**, not the plastic horn.

### Team kit path

Configure a servo as `arm_servo`. After start: move to three positions with short pauses. Add a challenge: button or gamepad bumper selects the next position.

## The Discovery

**What happened?** Positions are easier to reason about than “spin for a bit.”

**You built it!** You learned: discrete arm states for missions.

---

## Lab D: Driver station thinking

---

**The Goal:** Map controls to robot actions — a mini driver station.

### Practice path

**Parts:** Potentiometer and/or two buttons + motor driver from Lab B (or LED if you are still warming up).

| Control          | Robot action                         |
|------------------|--------------------------------------|
| Pot left / right | Motor reverse / forward (or LED dim) |
| Button A         | Arm to REACH (or LED pattern B)      |
| Button B         | Arm to STOW (or LED pattern A)       |

Read controls inside a forever loop. Print values to Serial / Mu so you can debug.

## Team kit path

Map gamepad left stick Y to `drive_motor` power (start with a low max). Map A/B buttons to two servo positions. Deadband: ignore tiny stick noise near center.

## The Discovery

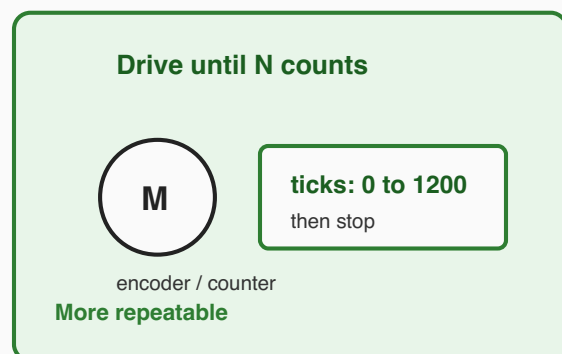
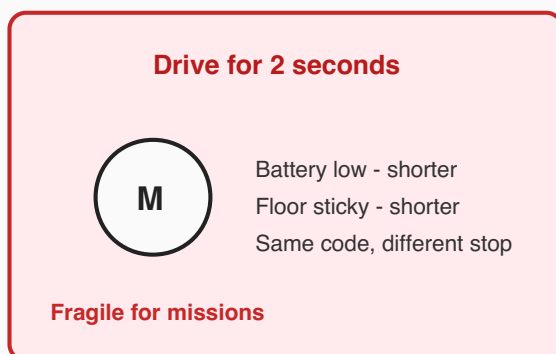
**Experiment:** Swap which button does REACH vs STOW. Can a partner drive without you explaining?

**You built it!** You learned: control map + loop + deadband thinking.

## Lab E: Move by counting

**The Goal:** See why timed drive is fragile — then use counters when you have them.

### Timed drive vs move-by-counting



## Practice path

1. Timed drive: run the motor for 2 seconds. Mark how far it went on the table (tape).
2. Change nothing in code. Repeat with a weaker battery, a finger drag on the shaft, or a slight uphill tilt (mentor-safe). Distance changes.
3. **Concept:** time  $\neq$  distance when load or voltage changes.

Classroom boards often lack wheel encoders. That is OK — the lesson is the failure mode. If you later get an encoder motor, count ticks instead of seconds.

## Team kit path

1. Use a motor with an encoder. Reset the counter, run to a target tick count at low power, then stop.
2. Challenge: run forward N ticks, then reverse to the start count.
3. Skip PID for now — low power and a generous stop window are enough.

## The Discovery

**What happened?** Counts aim at the same place more often than a blind timer.

**You built it!** You learned: missions prefer measuring motion, not only sleeping.

---

## Scratch / MakeCode / Python → robot program map

---

| Idea you already know                                               | Robot mission habit                 |
|---------------------------------------------------------------------|-------------------------------------|
| <code>when green flag clicked</code>                                | Set up / init, then wait for start  |
| <code>forever</code>                                                | Driver-control loop                 |
| Timed list of steps                                                 | Timed mission sequence              |
| Sprite / variable names                                             | Named motors, servos, sensors       |
| <code>if &lt;sensor&gt;</code>                                      | Gate motors and mission branches    |
| Custom blocks / functions                                           | Reuse “drive forward,” “arm reach”  |
| CircuitPython <code>while True</code> / Arduino <code>loop()</code> | Driver control on a classroom board |

---

## Mission ready checklist

---

Before you call this module done:

- [ ] I can explain brain/signals vs motor power
  - [ ] I can describe set up → wait for start → timed mission **or** driver loop
  - [ ] I keep a **named parts list** before coding
  - [ ] I drove a motor safely (transistor path or hub motor port)
  - [ ] I used a sensor to change robot behavior
  - [ ] I moved an arm (servo or named LED states)
  - [ ] I mapped controls to actions (pot/buttons or gamepad)
  - [ ] I can explain why timed drive is fragile
  - [ ] I wrote a short **build log**: what we tried, what broke, what we changed
- 

## What's next

---

1. **Invent**: a 30-second timed mission (three steps) **and** a driver mode for the same robot parts.
2. **Share** your parts list and build log with a mentor or teammates.
3. Revisit [Intermediate Circuits](#) if motor wiring feels shaky, or [CircuitPython](#) / [Arduino](#) for more text practice.

Mentors: see [docs/robot-missions-coach.md](#) for a skills map, meeting sequence, and hub doc links.